CSC3002
C/C++ Programming

String

Prof. Dr. Ruizhi Chen，SDS/SAI
Office: TxC 717
chenruizhi@cuhk.edu.cn

Strings

1 Using strings in C and C++
2 Characters and the <cctype> library
3 The C-style strings and the <cstring> library
4 Using strings as abstract values in C++
5 String operations
6 Modifying the contents of a string
7 Writing string applications

Introduction to the C++ Standard Libraries

- A collection of classes and functions, which are written in the core language and part of the C++ ISO Standard itself. Features of the C++ Standard Library are declared within the std namespace
  - Containers: vector, queue, stack, map, set, etc.
  - General: algorithm, functional, iterator, memory, etc.
  - Strings
  - Streams and Input/Output: iostream, fstream, sstream, etc.
  - Localization
  - Language support
  - Thread support library
  - Numerics library
  - C standard library: cmath, cctype, cstring, cstdio, cstdlib, etc.

Using strings in C and C++

- Text data nowadays are as important as numeric data. Almost any program that you write in any modern language is likely to use string data at some point, even if it is only to display instructions to the user or to label results.
- Text data processing is essential for Large Language Model
- Conceptually, a string is simply a sequence of characters, which is precisely how strings are implemented in C.
- As a newly designed language, especially one to extend C with the object-oriented programming paradigm, C++ supports a higher-level view of strings, as objects.
- The different strategies used by C and C++ on strings show the differences between different programming paradigms.

Characters

- Both C and C++ use ASCII (American Standard Code for Information Interchange) as their encoding for character representation. The data type char therefore fits in a single eight-bit byte.
- With only 256 possible characters, ASCII is inadequate to represent the many alphabets in use throughout the world. In most modern language, ASCII has been superseded by Unicode, which permits a much larger number of characters.
- Even though the weaknesses in the ASCII encoding were clear at the time C++ was designed, changing the definition of char was impossible given the decision to keep C as a subset.
- The C++ libraries define the type wchar_t to represent "wide characters" that allow for a larger range. The details of the wchar_t type are beyond the scope of this text. We will stick with the traditional char type.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

www.lookuptables.com

Source: www.LookupTables.com

128	Ç	144	É	160	á	176	▒	192	└	208	╨	224	α	240	≡
129	ü	145	æ	161	í	177	▒	193	┴	209	╤	225	ß	241	±
130	é	146	Æ	162	ó	178	▓	194	┬	210	╥	226	Γ	242	≥
131	â	147	ô	163	ú	179	│	195	├	211	╙	227	π	243	≤
132	ä	148	ö	164	ñ	180	┤	196	─	212	╘	228	Σ	244	⌠
133	à	149	ò	165	Ñ	181	╡	197	┼	213	╒	229	σ	245	⌡
134	å	150	û	166	ª	182	╢	198	╞	214	╓	230	µ	246	÷
135	ç	151	ù	167	º	183	╖	199	╟	215	╫	231	τ	247	≈
136	ê	152	ÿ	168	¿	184	╕	200	╚	216	╪	232	Φ	248	°
137	ë	153	Ö	169	⌐	185	╣	201	╔	217	┘	233	Θ	249	∙
138	è	154	Ü	170	¬	186	║	202	╩	218	┌	234	Ω	250	·
139	ï	155	¢	171	½	187	╗	203	╦	219	█	235	δ	251	√
140	î	156	£	172	¼	188	╝	204	╠	220	▄	236	∞	252	ⁿ
141	ì	157	¥	173	¡	189	╜	205	═	221	▌	237	φ	253	²
142	Ä	158	₧	174	«	190	╛	206	╬	222	▐	238	ε	254	■
143	Å	159	ƒ	175	»	191	┐	207	╧	223	▀	239	∩	255	

Source: www.LookupTables.com

www.LookupTables.com

Single Characters

- To use variables of the single character type:

```
char ch;
char ch = 'a';
char ch = 97;
```

- Exercise: What is the equivalent statement of the following:

```
char ch = 32;
```

```
char ch = ' ';
```

- Some characters are special control characters:

```
char ch = 9;  // '\t' (Tab)
char ch = 10; // '\n' (New line)
char ch = 13; // '\r' (Return)
```

- To break a line, Mac uses \r, Unix uses \n, and Windows uses \r\n.

The <cctype> (ctype.h) Interface

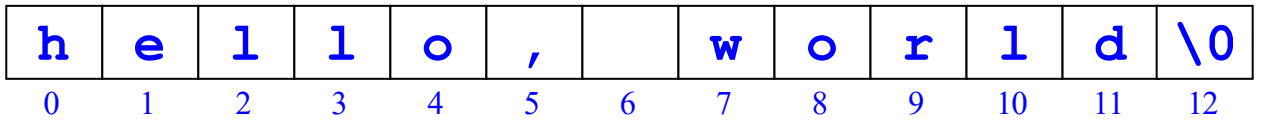
`bool isdigit(char ch)` Determines if the specified character is a digit.
`bool isalpha(char ch)` Determines if the specified character is a letter.
`bool isalnum(char ch)` Determines if the specified character is a letter or a digit.
`bool islower(char ch)` Determines if the specified character is a lowercase letter.
`bool isupper(char ch)` Determines if the specified character is an uppercase letter.
`bool isspace(char ch)` Determines if the specified character is ***whitespace*** (spaces and tabs).
`char tolower(char ch)` Converts **`ch`** to its lowercase equivalent, if any. If not, **`ch`** is returned unchanged.
`char toupper(char ch)` Converts **`ch`** to its uppercase equivalent, if any. If not, **`ch`** is returned unchanged.

The Legacy of C-Style Strings

- In C, we can represent strings, i.e., sequences of characters, as plain arrays of elements of a character type:

```
char cstr[10];
char cstr[] = "hello";
char cstr[] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

- If you put double quotation marks around a sequence of characters, you get what is called a C string literal (const char[] type). The characters are stored in an array of bytes, terminated by a null byte whose ASCII value is 0. E.g., the characters in the C string "hello, world" are arranged like this:



- Character positions in a C string are identified by an index that begins at 0 and extends up to one less than the length of the string.

The Legacy of C-Style Strings

- Question: is "a" the same as 'a'?

  "a" is a string literal containing an 'a' and a null terminator '\0' , and therefore is a 2-char array, ‘a’ is just a char.

Calling C String Functions

- The <cstring> (string.h) interface offers a lot of functions you can use to operate C strings. For example, if you want to determine the length of a C string cstr, you can use the following function strlen:

```
char cstr[] = "hello";
int len = strlen(cstr);
```

- If you want to assign a value to a C string cstr, you can use the following function strcpy:

```
strcpy(cstr, "world");
```

Does **cstr** *have enough space to hold* **"world"***?*

- However, you cannot directly assign a value to a C string except for initialization:

```
char cstr[] = "world";
```



```
cstr = "world";
cstr[] = "world";
```



The <cstring> Interface

Copying:

memcpy	Copy block of memory (function)
memmove	Move block of memory (function)
strcpy	Copy string (function)
strncpy	Copy characters from string (function)

Concatenation:

strcat	Concatenate strings (function)
strncat	Append characters from string (function)

Comparison:

memcmp	Compare two blocks of memory (function)
strcmp	Compare two strings (function)
strcoll	Compare two strings using locale (function)
strncmp	Compare characters of two strings (function)
strxfrm	Transform string using locale (function)

Searching:

memchr	Locate character in block of memory (function)
strchr	Locate first occurrence of character in string (function)
strcspn	Get span until character in string (function)
strpbrk	Locate characters in string (function)
strrchr	Locate last occurrence of character in string (function)
strspn	Get span of character set in string (function)
strstr	Locate substring (function)
strtok	Split string into tokens (function)

The <cstring> Example

```
#include <iostream>
#include <cstring>

int main() {
    char cstr[80];
    strcpy(cstr, "these ");
    strcat(cstr, "strings ");
    strcat(cstr, "are ");
    strcat(cstr, "concatenated.");
    std::cout << cstr << " length = " << strlen(cstr)<<std::endl;
    return 0;
}
```

```
>these strings are concatenated. length = 31
>
```

The <cstring> Exercise

Exercise: what is the output?

```
#include <iostream>
#include <cstring>
using std::cout; using std::endl;

int main() {
    char cstr[] = "hello";
    cout << cstr << endl
         << sizeof cstr << endl
         << strlen(cstr) << endl;
    strcpy(cstr, "hello world");
    cout << cstr << endl
         << sizeof cstr << endl
         << strlen(cstr) << endl;
    return 0;
}
```

sizeof x returns the actual memory size of variable **x**.



Using Strings as Abstract Values

- Ever since the very first program in the text, which displayed the message "hello, world" on the screen, you have been using strings to communicate with the user.

- Up to now, you have not had any idea how C++ represents strings inside the computer or how you might manipulate the characters that make up a string.

- For most applications, the abstract view of strings you have held up to now is precisely the right one. On the inside, strings are surprisingly complicated objects whose details are better left hidden.

Using Strings as Abstract Values

- As a newly designed language, especially one to extend C with the object-oriented programming paradigm, C++ supports a higher-level view of strings, as objects.
- A C++ library <string> provides a convenient abstraction for working with strings of characters by making string a class whose methods hide the underlying complexity.
- A class is the term for data types that support the object-oriented programming paradigm. The operations that apply to instances of a class are called methods.

Exercise

What are the meaning of the following statements?

```
#include <cstring>
// the C string library, used in C++
#include <string.h>
// the C string library, used in C, acceptable in C++
#include <string>
// the C++ string library
#include "cstring.h"
// user-defined cstring.h
#include "string.h"
user-defined string.h
#include <cstring.h>
// error even if you defined cstring.h by yourself
```



Classes and Objects

- In C, structures define the representation of a compound value, while functions define operations on these data. In C++, these two ideas are integrated.
- Object-oriented languages are characterized by representing most data structures as objects that encapsulate both the representation and the behavior of the objects (i.e., operations on the data) in a single entity.
- C++ object model is based on the idea of a class, which is a template describing all objects of a particular type. The class definition specifies the representation of the object by naming its fields or instance variables (e.g., compound property/attribute values) and the behavior of the object by providing a set of methods.
- New objects are created as instances of a particular class, and the creation of new instances is called instantiation.

Representing Points Using a Class

- Although structure types are part of the history of C++ and the languages that came before it, they have largely been supplanted by classes, which offer greater power and flexibility. The Point structure from the preceding section is identical to the following class definition:

```
class Point {
public:
    int x;
    int y;
    double length();
    Point operator+(Point& pt);
};
```

fields

Methods

- This definition allows you to declare a Point object like this:

```
Point pt;
```

- As the x and y fields are included as part of the public section, which is visible to clients, you can select the public fields using the dot operator (.), as in pt.x and pt.y.

The Hello Name Program

FIGURE 3-1 An interactive version of the "Hello World" program

```
/*
 * File: HelloName.cpp
 * -------------------
 * This program extends the classic "Hello world" program by asking
 * the user for a name, which is then used as part of the greeting.
 * This version of the program reads a complete line into name and
 * not just the first word.
 */

#include <iostream>
#include <string>
using namespace std;

int main() {
   string name;
   cout << "Enter your full name: ";
   cin >> name;
   cout << "Hello, " << name << "!" << endl;
   return 0;
}
```

Operators in the string Class

`str[i]` Returns the **`i`**th character of **`str`**. Assigning to **`str[i]`** changes that character.
`s1 + s2` Returns a new string consisting of **`s1`** concatenated with **`s2`**.
`s1 = s2;` Copies the character string **`s2`** into **`s1`**.
`s1 += s2;` Appends **`s2`** to the end of **`s1`**.
`s1 == s2` (and similarly for **`<`**, **`<=`**, **`>`**, **`>=`**, and **`!=`**) Compares to strings lexicographically.
`str.c_str()` **// this is a public member function** Returns a C-style character array containing the same characters as **`str`**.

- Unlike most languages, C++ allows classes to redefine the meanings of the standard operators. As a result, several string operations, such as + for concatenation, are implemented as operators (overloading).
- To convert the C++ string objects into C string literals, simply apply the c_str method to the C++ string.

The [ ] Operator of a string object

```
char &str::operator[](int idx)
{
        if (idx < maxlen && data[idx] != '\0') return data[idx];

        else throw std::out_of_range("index out of range");
}
```

```
str s = “Test”;

s[0] = ‘W’; // s = “West”;

char c = s[0]; // c = ’W’
```

Concatenation of Strings

- As those of you who have studied Python already know, the + operator is a convenient shorthand for concatenation for the string objects, which consists of combining two strings end to end with no intervening characters.
- In Python, the + operator is often used to combine items as part of a print call, as in

```
print("Hello, " + name + "!");
```

- In C++, you achieve the same result using the << operator:

```
cout << "Hello, " << name << "!" << endl;
```

- Although you might imagine otherwise, you can't always use the + operator in this statement, depending on whether name is a C string (array) or a C++ string object.

```
string name = "World";

cout << "Hello, " + name + "!" << endl;
```

OK

String literal vs. C++ string object

- C++ automatically converts a C string literal to a C++ string object whenever the compiler can determine that what you want is a C++ string object:

```
string str = "hello, world";
```

- By contrast, C++ does not allow you to write the declaration:

```
string str = "hello" + ", " + "world";
```



- The + operator cannot be applied to C string literals. To get around this problem, you can explicitly convert a string literal to a string object by calling string on the literal:

```
string str = string("hello") + ", " + "world";
```

```
string str = "hello" + string(", ") + "world";
```

```
string str = "hello" + ", " + string("world");
```



String literal vs. Char array vs. String object

Exercise: what is the output?

```
#include <iostream>
#include <string>

int main() {
    char name1[] = "Ray";
    std::string name2 = "Ray";
    std::cout << "Hello, " << name1 << std::endl;
    std::cout << "Hello, " << name2 << std::endl;
    std::cout << "Hello, " + name1 << std::endl;
    std::cout << "Hello, " + name2 << std::endl;
    return 0;
}
```



Common Methods in the string Class

`str.length()` Returns the number of characters in the string **`str`**.
`str.at(index)` Returns the character at position **`index`**; most clients use **`str[index]`** instead.
`str.substr(pos, len)` Returns the substring of **`str`** starting at **`pos`** and continuing for **`len`** characters.
`str.find(ch, pos)` Returns the first index ≥ **`pos`** containing **`ch`**, or **`string::npos`** if not found.
`str.find(text, pos)` Similar to the previous method, but with a string instead of a character.

Calling String Methods

- Calling a method of an object:

```
object.method(arguments);
```

- For example, if you want to determine the length of a string str (a string object), you can call the method as follows:

```
int len = str.length();
```

- You might also be tempted to use the strlen function we have used before with the C string literal:

```
int len = strlen(str);
```



Calling String Methods

Exercise: how do you determine the length of a string str?

- The object-oriented version:

```
int len = str.length();
```

- If you must use the strlen function from C:

```
int len = strlen(str.c_str());
```

The <string> Library Example

```
#include <iostream>
#include <cstring>

int main() {
    char str[80];
    strcpy(str, "these ");
    strcat(str, "strings ");
    strcat(str, "are ");
    strcat(str, "concatenated.");
    std::cout << str << " length = " << strlen(str);
    return 0;
}
```

C string literal

```
#include <iostream>
#include <string>

int main() {
    std::string str;
    str = "these ";
    str = str + "strings " + "are " + "concatenated.";
    std::cout << str << " length = " << str.length();
    return 0;
}
```

C++ string

Strings as an Abstract Data Type

- Because C++ includes everything in its predecessor language, C strings are a part of C++, and you will occasionally have to recognize that this style of string exists.
- For almost every program you write, it will be far easier to use the C++'s string class, which implements strings as an abstract data type, which is defined by its behavior rather than its representation. All programs that use the string class must include the <string> library interface.

Selecting characters in strings

- The <string> library offers two different mechanisms for selecting individual characters from a string:

```
str[index]
```

```
str.at(index)
```

- The only difference is that at checks to make sure the index is in range, 0~str.length()-1.
- Both methods can be used to assign a new value to the character:

```
str[index] = 'H';
```

```
str.at(index) = 'H';
```

- The former has better readability while the latter has range-checking.

Iterating through the characters in strings

- When you work with strings, one of the most important patterns involves iterating through the characters in a string, which requires the following code:

```
for (int i = 0; i < str.length(); i++) {
   . . . body of loop that manipulates str[i] . . .
}
```

```
for (int i = str.length() - 1; i >= 0; i--)
```

- The following function reverses the argument string so that, calling reverse("desserts") returns "stressed":

```
string reverse(string str) {
   string rev = "";
   for (int i = str.length() - 1; i >= 0; i--) {
      rev += str[i];
   }
   return rev;
}
```

Modifying the Contents of a String

- In many languages, including Python, Java, C#, strings are immutable, which means that they never change once they are allocated.
- C++, by contrast, allows clients to change the contents of a string, both by assigning new values to selected characters and by calling string methods such as the following:

`str.erase(pos, count)` Deletes **`count`** characters from **`str`** starting at position **`pos`**.	←*Destructively changes* **`str`**
`str.insert(pos, text)` Inserts the characters from **`text`** into **`str`** starting at position **`pos`**.	←*Destructively changes* **`str`**
`str.replace(pos, count, text)` Replaces **`count`** characters in str with **`text`** starting at position **`pos`**.	←*Destructively changes* **`str`**

Modifying the Contents of a String

- As a tool for writing programs that are easier to debug and maintain, immutable strings have many advantages over their modifiable counterparts in C++. Fortunately, it is easy to secure these advantages in C++ by avoiding the use of destructive operations like erase, insert, replace, and assignment to individual characters.
- Example: Case conversion without changing the original (safe, but is it efficient?)

```
string toUpperCase(string str) {
   string result = "";
   for (int i = 0; i < str.length(); i++) {
      result += toupper(str[i]);
   }
   return result;
}
```

Avoiding the use of destructive operations

- Example: Case conversion both safely and efficiently (why?)

```
string toUpperCase(string str) {
   for (int i = 0; i < str.length(); i++) {
      str[i] = toupper(str[i]);
   }
   return str;
}
```

- Exercise: Case conversion in place (most efficient but not safe, why?)

```
void toUpperCaseInPlace(string & str) {
   for (int i = 0; i < str.length(); i++) {
      str[i] = toupper(str[i]);
   }
}
```

- Question: can we implement reverse the same way?

Exercise: Recognizing Palindromes

- A palindrome is a word that reads identically backward and forward, such as “level” or “noon”.
- Write a C++ program isPalindrome that checks whether a string is a palindrome.

```
bool isPalindrome(string str) {
    int n = str.length();
    for (int i = 0; i < n / 2; i++) {
        if (str[i] != str[n - i - 1]) return false;
    }
    return true;
}
```

```
bool isPalindrome(string str) {
    return str == reverse(str);
}
```

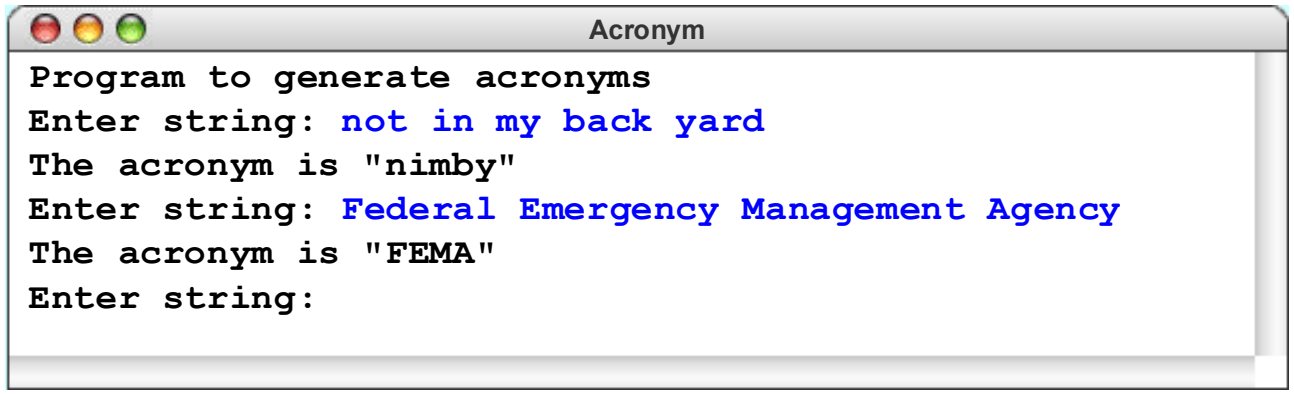
- Efficiency vs. Readability

Exercise: Writing String Applications

- An acronym is a word formed by taking the first letter of each word in a sequence, as in

  "self-contained underwater breathing apparatus" → "scuba"

- Write a C++ program that generates acronyms, as illustrated by the following sample run:





The End